Embedded Data Structure
(Lecture Note 13)

Tree (2)

2025. 4. 15

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of Last Class

KU KONKUK UNIVERSITY

- **Tree with pointers**
  - **Remember the tree representation with double linked list**
    - Two links : left and right
    - Tree is initialized with head and tail (all the nodes indicate tail)
  - **Traverse your tree with recursion**
    - Four different strategies for traversing a given tree

      1) Pre-order : visit the root → left sub-tree → right sub tree
      2) In-order : left sub-tree → visit the root → right sub tree

      ※ Other two strategies will be introduced in this class

Post-order Traverse (1/4)

KU KONKUK UNIVERSITY

▪ Basic concept of post-order traverse

- **Pseudo code :**

1. Visit the left subtree
2. Visit the right subtree
3. Visit the root

Main functions :

*** Push "all the left child" node**

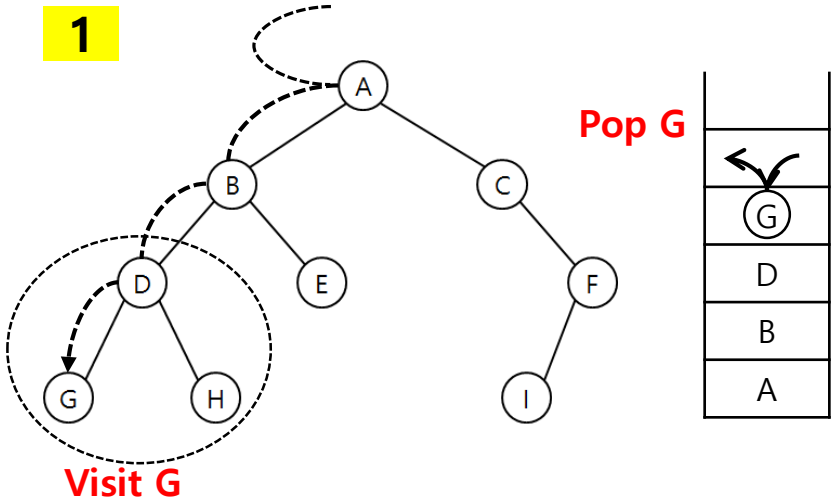
*** Visit the "right child" with pop & push**

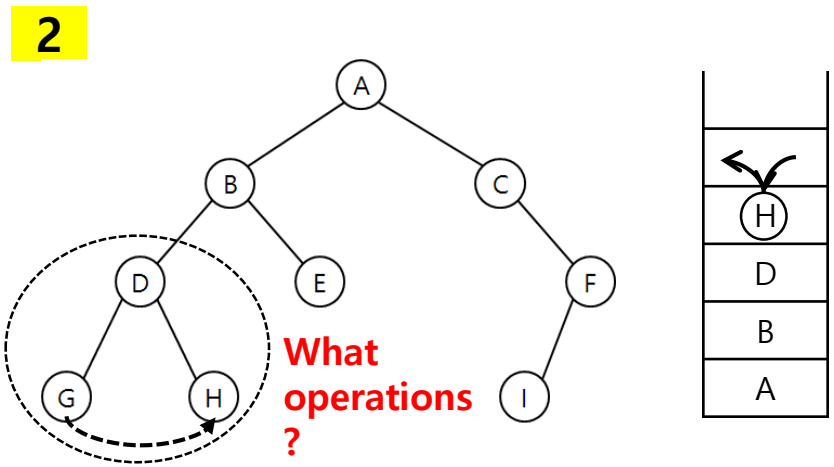
```
void postorder_traverse(node *t)
{
    if(t != tail)
    {
        postorder_traverse(t->left);
        postorder_traverse(t->right);
        visit(t);
    }
}
```

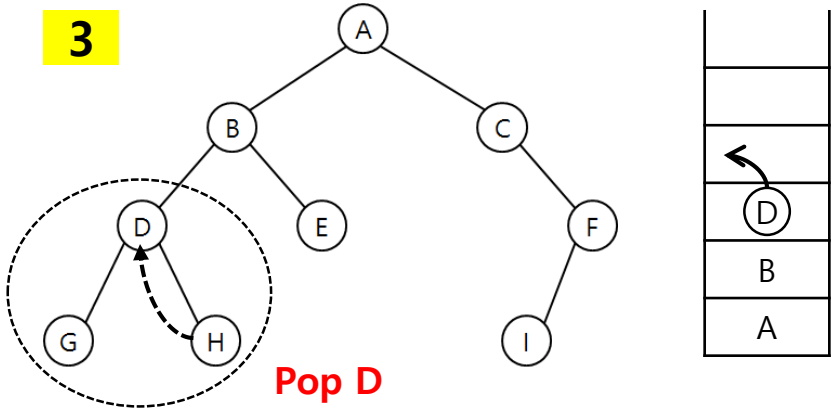
➔ Think about "recursive" operation

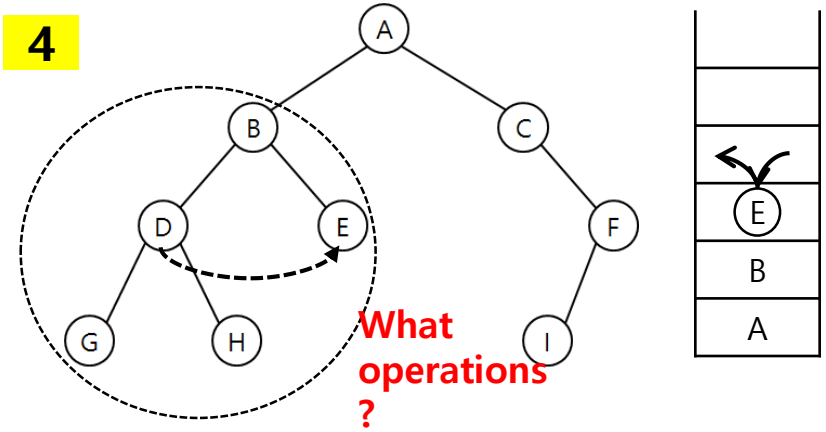
Post-order Traverse (2/4)

- **Traverse example**



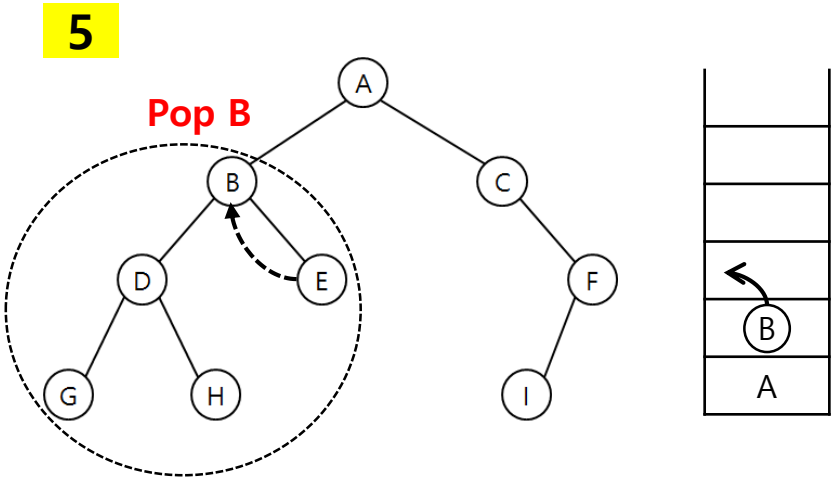


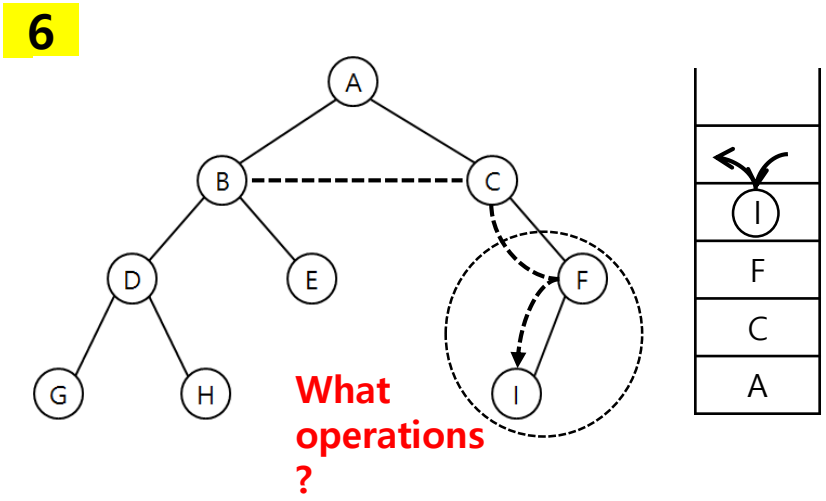


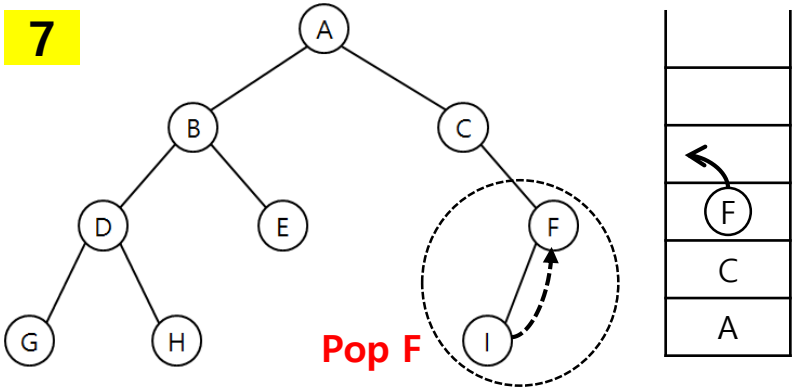


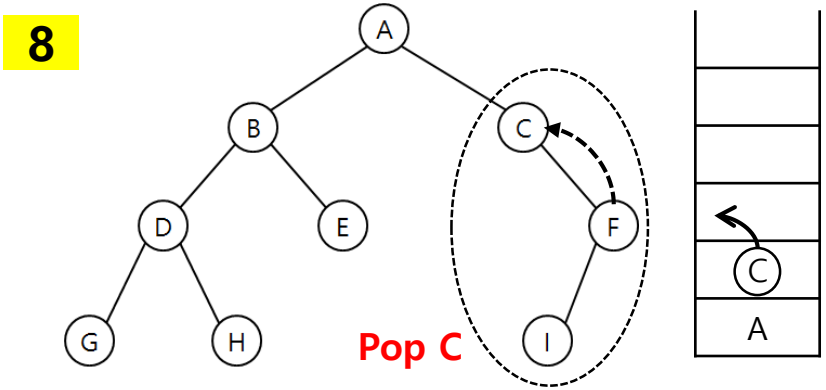
Post-order Traverse (3/4)

- **Traverse example – cont'd**



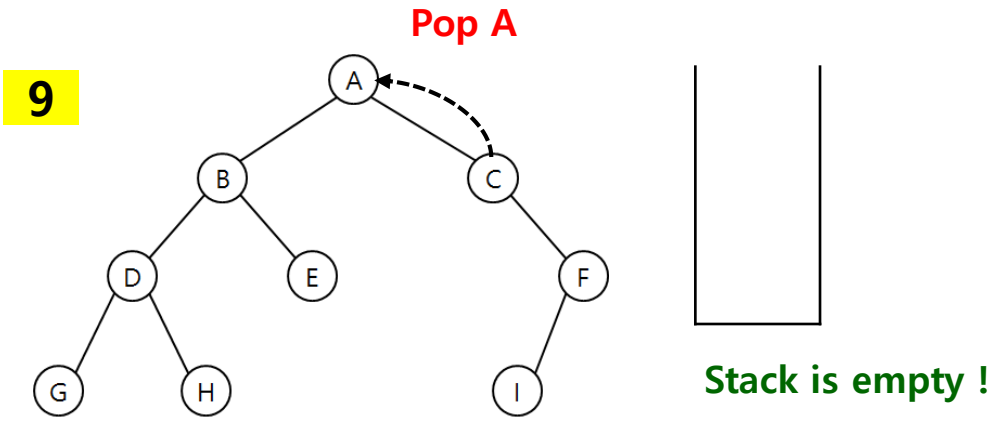






Post-order Traverse (4/4)

- **Traverse example – cont'd**
  - **Now, we have visited all the node**



*** Traverse result : G→H→D→E→B→I→F→C→A**

Level-order Traverse (1/4)

- **Basic concept of level-order traverse**
  - **Pseudo code :**

1. Put the root in the queue
2. If the queue is not empty,
   2.1. t=get() and visit "t"
   2.2. If t has a left child, then put it
   2.3. If t has a right child, then put it
3. Finish (queue is empty)

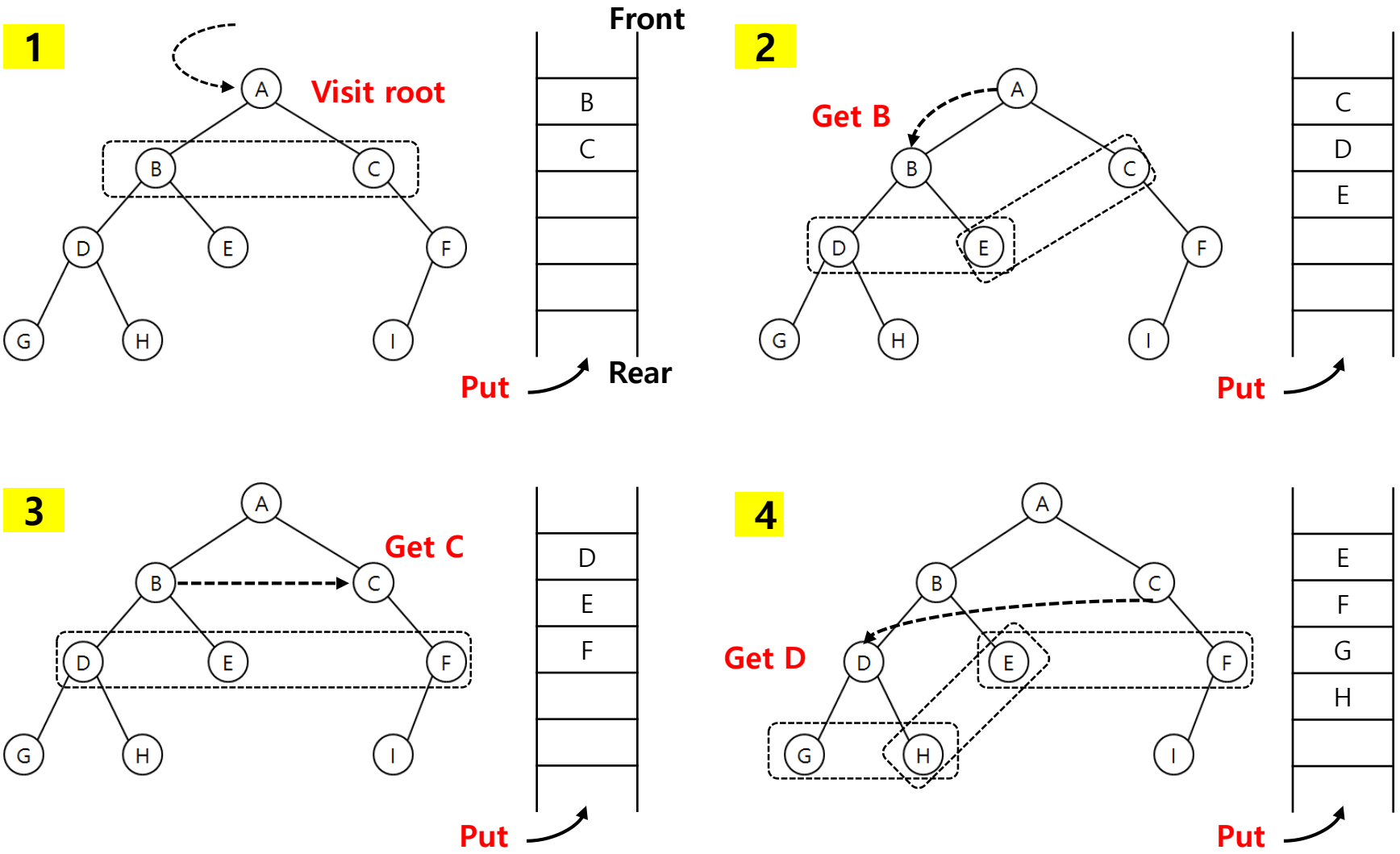
```
void levelorder_traverse(node *t)
{
    put(t);

    while(!is_queue_empty())
    {
        t = get();
        visit(t);

        if(t->left != tail)
            put(t->left);
        if(t->right != tail)
            put(t->right);
    }
}
```

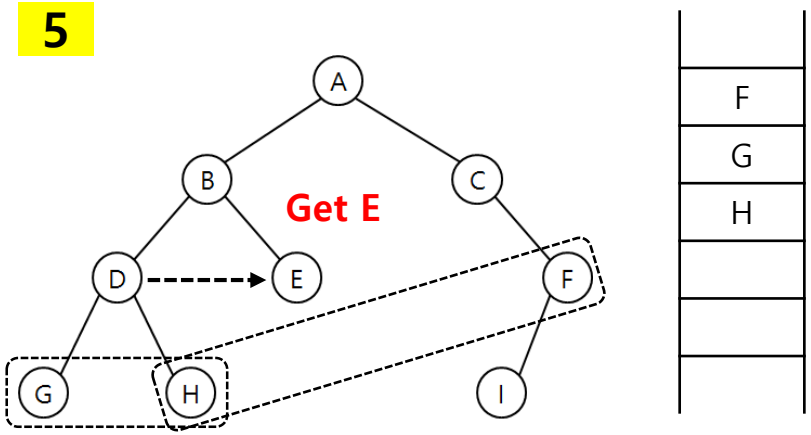
Level-order Traverse (2/4)

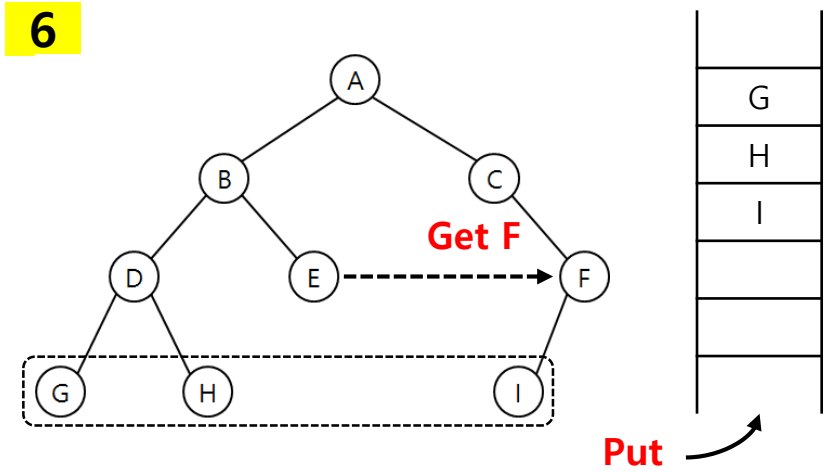
- **Traverse example**

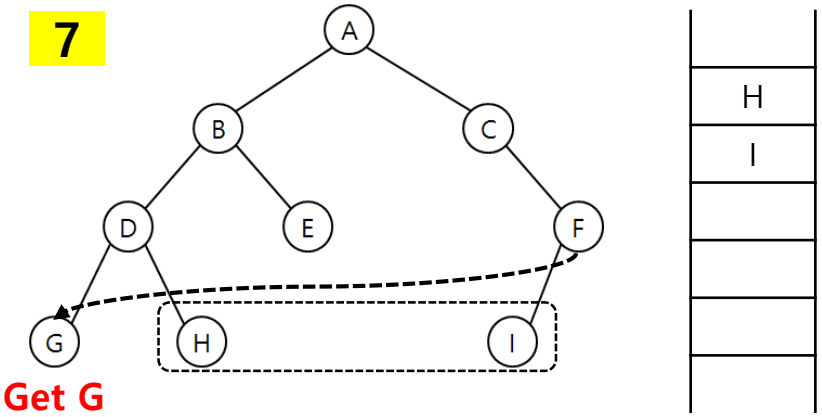


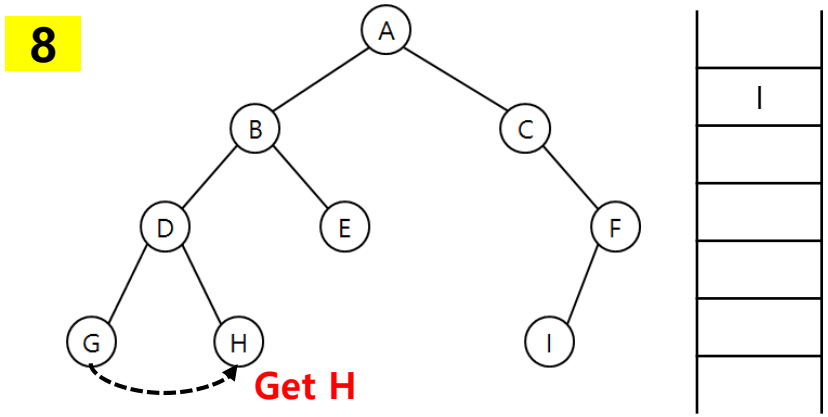
Level-order Traverse (3/4)

- **Traverse example – cont'd**



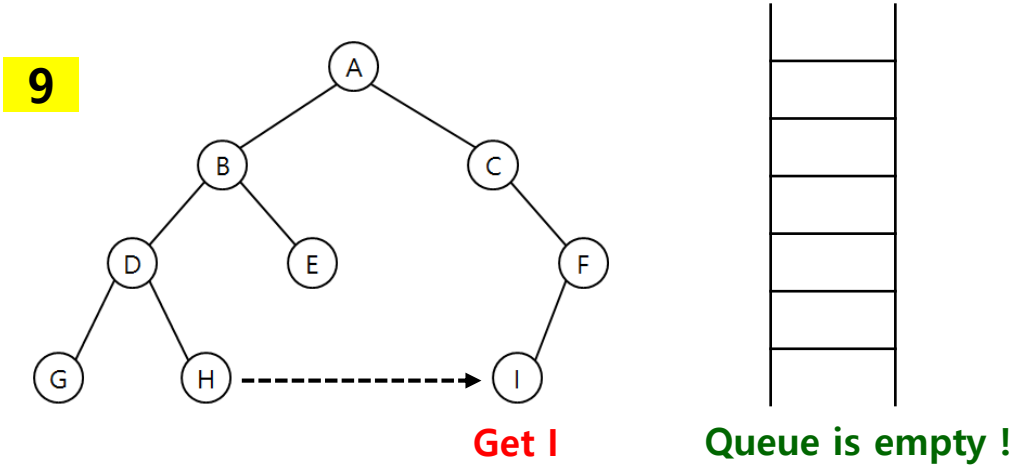






Level-order Traverse (4/4)

- **Traverse example – cont'd**
  - **Now, we have visited all the node**

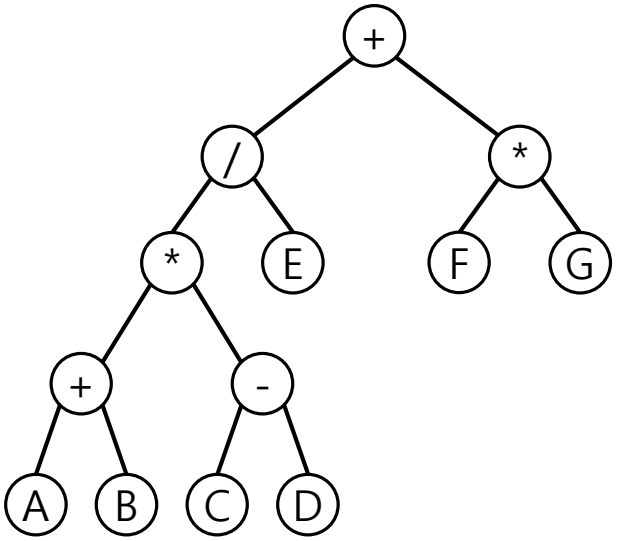


*** Traverse result : A→B→C→D→E→F→G→H→I**

Application : Postfix Tree (1/3)

- **Parse tree : postfix equation representation**

((((A+B) * (C-D)) / E) + (F*G))



[Example : infix notation]

Pseudo code :

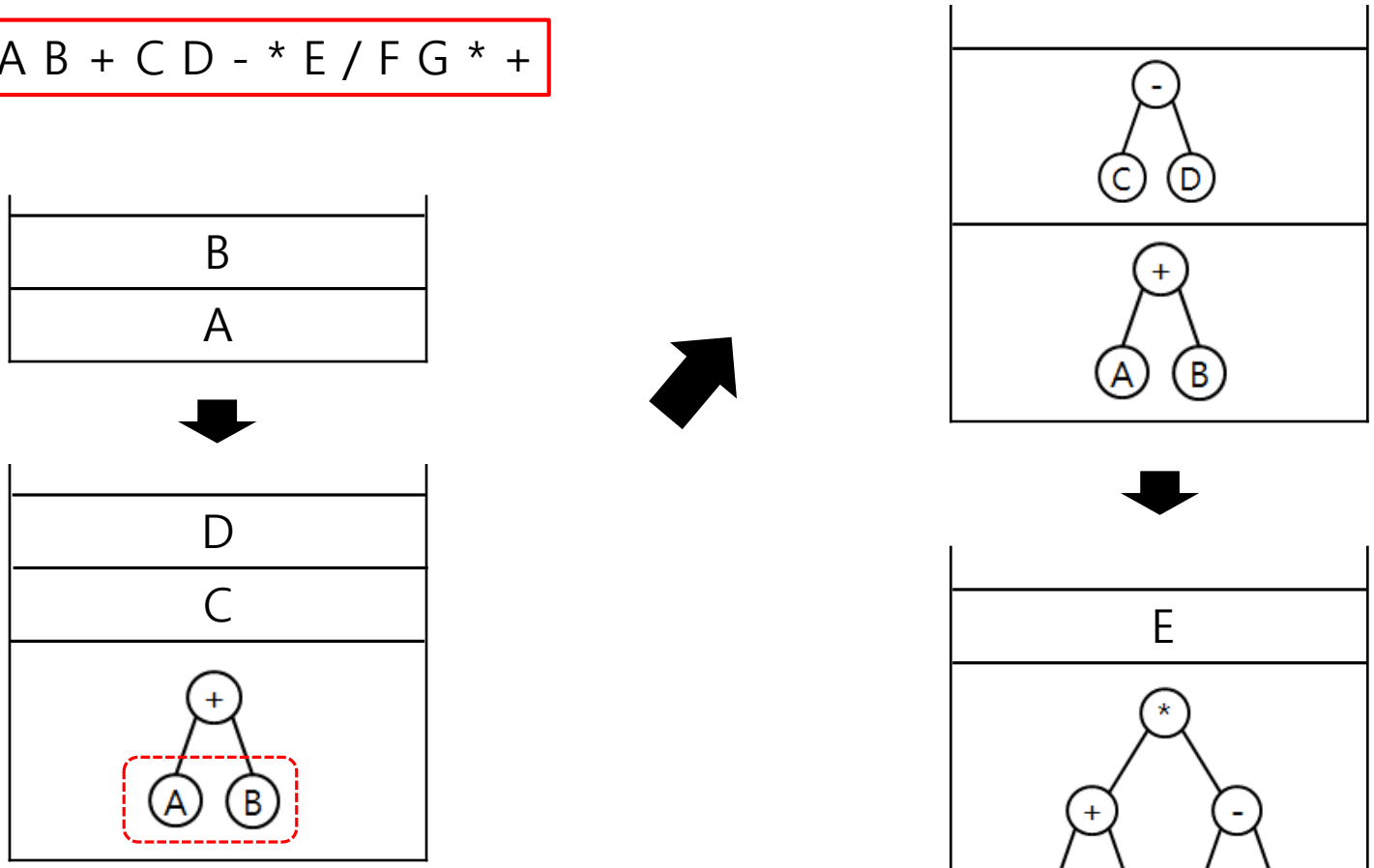
(Make a postfix tree)

1. When you meet operands,
   make a node and push it to the stack
2. When you meet the operators,
   2.1. Pop and make a right child
   2.2. Pop and make a left child
   2.3. After that, push the operator
3. Last node in the stack becomes root

Application : Postfix Tree (2/3)

- **Parse tree : postfix equation representation**

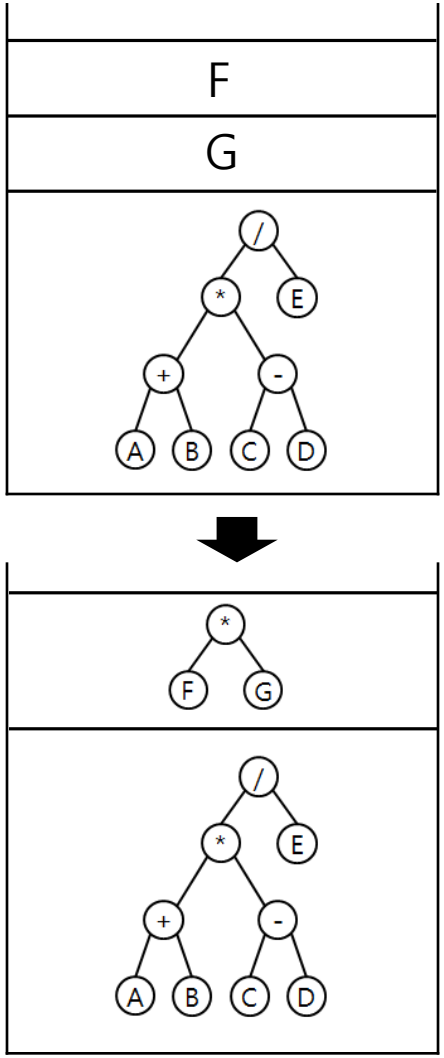
A B + C D - * E / F G * +

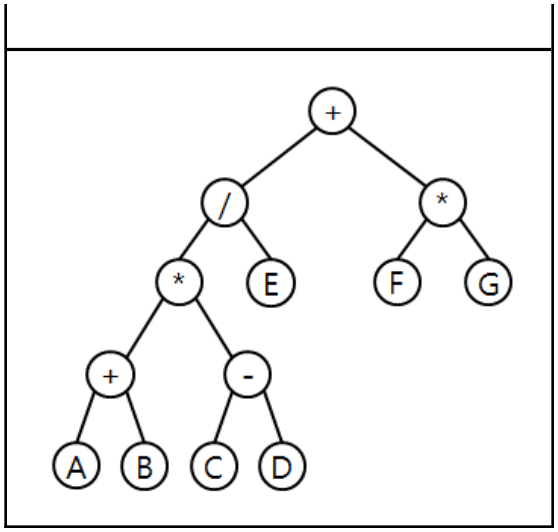


A and B are not stored in the stack (just for better understanding)

Application : Postfix Tree (3/3)

- **Parse tree : postfix equation representation**





[Parse tree (postfix)]

Now, we are ready to traverse trees with C programming !

Implementation (1/4)

- **Make parse tree & traverse (full source code) – cont'd**

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

typedef struct _node{
   int key;
   struct _node *left;
   struct _node *right;
}node;

node *head, *tail;
node *stack[MAX];
node *queue[MAX];
int top, front, rear;

void init_stack() {
   top = -1;
}

void init_queue() {
   front = rear = 0;
}
```

```
int is_stack_empty(){
   return (top == -1);
}

int is_queue_empty(){
   return (front == rear);
}

node *push(node *t){
   stack[++top] = t;
   return t;
}

node *pop() {
   return stack[top--];
}

node *put(node *t){
   queue[rear] = t;
   rear = ++rear%MAX;
   return t;
}
```

```
node *get() {
   node *t;
   t = queue[front];
   front = ++front%MAX;
   return t;
}

void init_tree() {
   head = (node *)calloc(1, sizeof(node));
   tail = (node *)calloc(1, sizeof(node));
   head->left = tail;
   head->right = tail;
   tail->left = tail;
   tail->right = tail;
}

int is_operator(int k) {
   return (k == '+' || k == '-' || ~~);
}

void visit(node *t) {
   printf("%c ", t->key);
}
```

Implementation (2/4)

- **Generation of parse tree**
  - **Input : postfix notation / Output : root node**

```
node *make_parse_tree(char *p)
{
    node *t;
    while(*p){
        while(*p == ' ') p++;
        t = (node *)calloc(1, sizeof(node));
        t->key = *p;
        t->left = tail;
        t->right = tail;
        if(is_operator(*p)){
            [                    ]
            [                    ]
        }
        [                ]
        p++;
    }
    return pop();
}
```

Think about it

When ?

When ?

Implementation (3/4)

- **Four different traverse functions**
  - **Recursive functions for pre-/in-/post-order (queue for level-order)**

```
void preorder_traverse(node *t)
{
    if(t != tail)
    {
        visit(t);
        preorder_traverse(t->left);
        preorder_traverse(t->right);
    }
}
```

```
void inorder_traverse(node *t)
{
    if(t != tail)
    {
        inorder_traverse(t->left);
        visit(t);
        inorder_traverse(t->right);
    }
}
```

※ Other two functions are explained in this lecture note

Implementation (4/4)

- **Main function**

```
void main()
{
    char post[256];
    init_stack();
    init_queue();
    init_tree();

    while(1){
        printf("₩n₩nInput postfix expression → ");
        gets(post);

        if(*post == NULL) break;

        head->right = make_parse_tree( [          ] );

        printf("₩nPre-order traverse → ");
        preorder_traverse(head->right);
    }
}
```

Summary

- **Basic concept of tree and a methodology to visit it**
  - **We traverse tree with pointers !**
- **Think about the applications for tree**
  - **Practice yourself with parsing tree example**